# Flow Map Matching Implementation Report

Replication of [**?**]

Deep Learning Project

February 11, 2026

## 1 Introduction

This project implements the framework introduced in *"Flow Map Matching with Stochastic Interpolants"* (arXiv:2406.07507). While traditional Flow Matching (FM) and Diffusion models define a generative process via a time-dependent velocity field $v_t(x)$, they require solving an Ordinary Differential Equation (ODE) during inference. This numerical integration, typically performed via Euler or Runge-Kutta solvers, necessitates multiple network evaluations, leading to significant computational overhead.

The core objective of this work is to learn the **Flow Map** $X_{s,t}(x)$ directly. Formally, given a velocity field $v$, the flow map is the unique solution to the initial value problem:

$$\frac{d}{dt}X_{s,t}(x) = v(X_{s,t}(x), t), \quad X_{s,s}(x) = x \tag{1}$$

By parameterizing $X_{s,t}(x)$ as a neural network, we transition from an Eulerian description of the flow to a Lagrangian one. This implementation demonstrates how the **Lagrangian Flow Map Matching (L-FMM)** objective allows for:

- **Single-Step Synthesis:** Mapping a sample from the source distribution $p_0$ (Gaussian noise) to the target manifold $p_1$ (Two Moons) in a single forward pass.
- **Mathematical Consistency:** Enforcing the algebraic properties of flows, specifically the *identity* $X_{t,t}(x) = x$ and the *semi-group* property $X_{s,t}(x) = X_{u,t}(X_{s,u}(x))$.
- **Efficiency:** Distilling the knowledge of a pre-trained "Teacher" velocity field into a fast "Student" flow map.

Through this implementation, we verify that the learned map effectively approximates the optimal transport paths while maintaining high fidelity to the underlying data distribution.

## 2 Methodology

Our implementation follows a two-stage distillation process, leveraging the flexibility of the stochastic interpolant framework.

## 2.1 Stage 1: Velocity Field (Teacher)

The teacher network learns the dynamics of the probability path. We consider a general interpolant of the form:

$$x_t = \alpha_t x_0 + \beta_t x_1 + \gamma_t z, \quad z \sim \mathcal{N}(0, I) \tag{2}$$

where $\alpha_t, \beta_t$ are scalar differentiable functions satisfying the boundary conditions $\alpha_0 = 1, \beta_0 = 0$ (source) and $\alpha_1 = 0, \beta_1 = 1$ (target). We also have the condition $\gamma_0 = \gamma_1 = 0$.

### 2.1.1 Linear Interpolant

In our primary experiments, we set $\gamma_t = 0$ to achieve **Rectified Flow** [3]. This choice is deeply rooted in Optimal Transport theory. We point out that there is no stochasticity in this framework; we focus exclusively on ODEs.

Generally, a deterministic conditional interpolant between a source $x_0$ and a target $x_1$ is defined as a time-dependent mixture:

$$x_t = \alpha_t x_0 + \beta_t x_1 \tag{3}$$

The corresponding conditional vector field is given by the time derivative:

$$u_t(x|x_0, x_1) = \dot{\alpha}_t x_0 + \dot{\beta}_t x_1$$

According to **Brenier's Theorem** [5], for the quadratic cost $c(x, y) = \frac{1}{2}\|x - y\|^2$, the optimal transport map is the gradient of a convex function. In our primary experiments, we selected the specific parametrization $\alpha_t = 1 - t$ and $\beta_t = t$ to achieve **Rectified Flow**. This choice corresponds to the displacement interpolant in Optimal Transport (for the 2-Wasserstein metric) and simplifies the trajectory to a straight line:

$$x_t = (1 - t)x_0 + tx_1, \quad \frac{dx_t}{dt} = x_1 - x_0 \tag{4}$$

The teacher $v_\theta(x, t)$ is then trained via Conditional Flow Matching (CFM) to approximate this constant velocity field.

This parametrization is "efficient" as it minimizes the kinetic energy $\int_0^1 \|\frac{dx_t}{dt}\|^2 dt$.

The teacher network $v_\theta(x, t)$ is trained using **Conditional Flow Matching** (CFM) [2]. The objective is to regress the vector field $v_\theta$ onto a target conditional vector field $u_t(x|x_0, x_1)$ that generates the probability path. The general loss function is defined as:

$$\mathcal{L}_{CFM}(\theta) = E_{t,x_0,x_1}\left[||v_\theta(x_t, t) - u_t(x|x_0, x_1)||^2\right]$$

For a general interpolant $x_t = \alpha_t x_0 + \beta_t x_1$ (we recall that $\gamma_t = 0$, the target velocity field corresponds to the time derivative of the path:

$$u_t(x|x_0, x_1) = \frac{d}{dt}x_t = \dot{\alpha}_t x_0 + \dot{\beta}_t x_1 \tag{5}$$

In our implementation, we employ the **Linear Interpolant** (Optimal Transport geodesics), defined by $\alpha_t = 1 - t$ and $\beta_t = t$. Consequently, the time derivatives become constants $\dot{\alpha}_t = -1$ and $\dot{\beta}_t = 1$, leading to the simplified regression target used in our code: $u_t = x_1 - x_0$, hence

$$\mathcal{L}_{CFM}(\theta) = E_{t,x_0,x_1}[||v_\theta(x_t, t) - (x_1 - x_0)||^2] \tag{6}$$

### 2.1.2 Alternative Parametrizations

While straight lines are optimal for the quadratic cost, the framework supports other interpolants depending on the desired properties of the probability path:

1. **Trigonometric (Variance Preserving):** Used commonly in Diffusion Models, this path follows $x_t = \cos(\frac{\pi}{2}t)x_0 + \sin(\frac{\pi}{2}t)x_1$. It keeps the variance of $x_t$ constant if $x_0$ and $x_1$ are unit variance, which can stabilize training in high dimensions.

2. **Stochastic (Brownian Bridge):** The framework is designed to support stochasticity, leveraging the theory of **Stochastic Interpolants** introduced by Albergo and Vanden-Eijnden [4]. By setting $\gamma_t = \sigma\sqrt{t(1-t)}$, we add a noise term that vanishes at both $t = 0$ and $t = 1$. This transforms the ODE into a Stochastic Differential Equation (SDE), allowing the model to explore multiple paths between the same pair of points.

### 2.1.3 Stochastic Interpolant (Brownian Bridge)

We extended the framework to support stochasticity by defining a non-zero noise coefficient $\gamma_t > 0$. We implemented a **Brownian Bridge** interpolant defined by $\gamma_t = \sigma\sqrt{t(1-t)}$, where $\sigma$ controls the diffusion scale. The position and velocity targets become:

$$x_t = (1-t)x_0 + tx_1 + \gamma_t z \tag{7}$$

$$u_t = (x_1 - x_0) + \dot{\gamma}_t z \tag{8}$$

where $z \sim \mathcal{N}(0, I)$ and $\dot{\gamma}_t = \sigma\frac{1-2t}{2\sqrt{t(1-t)}}$. In this regime, the target velocity $u_t$ is stochastic. Consequently, the teacher network $v_\theta(x, t)$ learns the conditional expectation of the drift $E[u_t|x_t]$, resulting in a non-zero irreducible loss corresponding to the variance of the injected noise.

## 2.2 Stage 2: Flow Map (Student)

We train a student network $X_\phi(x, s, t)$ to distill the teacher's knowledge into a direct mapping.

### 2.2.1 Lagrangian Distillation

We minimize the **Lagrangian Flow Map Matching (L-FMM)** loss, forcing the student's temporal derivative to match the teacher's predicted velocity:

$$\mathcal{L}_{L-FMM}(\phi) = E_{s,t,x_s}[||\partial_t X_\phi(x_s, s, t) - v_\theta(X_\phi(x_s, s, t), t)||^2]$$

### 2.2.2 Architectural Hard Constraints

To ensure mathematical validity, we use a "Hard Constraint" architecture. This guarantees the identity property $X(x, t, t) = x$ by construction, regardless of the weights:

$$X_\phi(x, s, t) = x + (t - s) \cdot \mathrm{Net}_\phi(x, s, t) \tag{9}$$

This formulation simplifies the learning task, as the network $\mathrm{Net}_\phi$ parameterizes the conditional displacement vector, implicitly modeling the effective velocity of the probability flow over the interval $[s, t]$.

# 3 Neural Architectures

The framework relies on two distinct architectures designed to handle continuous-time dynamics through sinusoidal embeddings.

## 3.1 Time Embedding Layer

To allow the networks to resolve high-frequency temporal variations, we use a Fourier feature mapping:

$$\text{Emb}(t) = [\sin(\omega_1 t), \cos(\omega_1 t), \ldots, \sin(\omega_k t), \cos(\omega_k t)] \tag{10}$$

where $\omega_i$ are log-linearly spaced frequencies. This embedding is concatenated with the spatial coordinates $x$.

## 3.2 Teacher and Student Specifications

- **Teacher ($v_\theta$):** A Multi-Layer Perceptron (MLP) with 3 hidden layers of 128 units and SiLU activations. It maps $R^d \times R \to R^d$, where $d = 2$ represents the dimensionality of the target manifold (Two Moons).
- **Student ($X_\phi$):** An MLP with similar depth but taking two time parameters $(s, t)$ as input. Its output is constrained by the identity-preserving wrapper:

$$X_\phi(x, s, t) = x + (t - s) \cdot \text{MLP}_\phi(x, \text{Emb}(s), \text{Emb}(t)) \tag{11}$$

# 4 Algorithm and Implementation

The practical implementation relies on stochastic optimization to approximate the continuous expectations. We minimize the loss functions using the **Adam** optimizer over mini-batches $\mathcal{B}$.

## 4.1 Phase 1: Teacher Training (Flow Matching)

The teacher learns the vector field by minimizing the Conditional Flow Matching objective.

**Algorithm 1:** Conditional Flow Matching (Teacher)

**Initialize** network $v_\theta$ and Adam optimizer.
**Define** coefficients $\alpha_t, \beta_t, \gamma_t$ and derivatives $\dot{\alpha}_t, \dot{\beta}_t, \dot{\gamma}_t$.
**while** not converged **do**
  Sample batch $\mathcal{B} = \{(x_{0,i}, x_{1,i})\}_{i=1}^N$.
  Sample time $t_i \sim \mathcal{U}(0, 1)$ and noise $z_i \sim \mathcal{N}(0, I)$.
  **Interpolation:** $x_{t,i} = \alpha_{t_i} x_{0,i} + \beta_{t_i} x_{1,i} + \gamma_{t_i} z_i$.
  **Target:** $u_{t,i} = \dot{\alpha}_{t_i} x_{0,i} + \dot{\beta}_{t_i} x_{1,i} + \dot{\gamma}_{t_i} z_i$.
  **Loss:** $\hat{\mathcal{L}}(\theta) = \frac{1}{N} \sum_{i=1}^N ||v_\theta(x_{t,i}, t_i) - u_{t,i}||^2$.
  Update $\theta$ using $\nabla_\theta \hat{\mathcal{L}}$.
**end while**

### 4.2 Phase 2: Student Distillation (L-FMM)

The student learns the flow map via Lagrangian distillation.

**Algorithm 2:** Lagrangian Map Distillation (Student)

**Load** pre-trained teacher $v_\theta$ (frozen).
**while** not converged **do**
  Sample batch $\mathcal{B}$ of size $N$.
  Sample $s_i, t_i \sim \mathcal{U}(0, 1)$.
  Construct valid path points $x_{s,i}$ via interpolation.
  **Prediction:** $x_{t,i} = X_\phi(x_{s,i}, s_i, t_i)$.
  **Differentiation:** Compute $\partial_t X_\phi$ via autograd.
  **Guidance:** $v_{target,i} = v_\theta(x_{t,i}, t_i)$.
  **Loss:** $\hat{\mathcal{L}}(\phi) = \frac{1}{N} \sum_{i=1}^{N} ||\partial_t X_\phi^{(i)} - v_{target,i}||^2$.
  Update $\phi$ using $\nabla_\phi \hat{\mathcal{L}}$.
**end while**

# 5 Results

### 5.1 Quantitative Evaluation

We evaluated the trained Stochastic Student ($\sigma = 0.1$) on three key physical properties. The exceptionally low error rates demonstrate that the model successfully distilled the Probability Flow ODE from the noisy teacher.

| **Metric** | **MSE Value** | **Status** |
|---|---|---|
| Identity Error ($X_{t,t}(x) \approx x$) | **0.000000** | **Excellent** |
| Semigroup Error (Composition) | **0.002631** | **Good** |
| ODE Consistency (vs Solver) | **0.003899** | **Good** |

Table 1: Validation Metrics on Test Set (N=2000). The low ODE Consistency error confirms that the Student effectively filtered out the Brownian noise to capture the underlying deterministic drift.

## 5.2 Visual Results

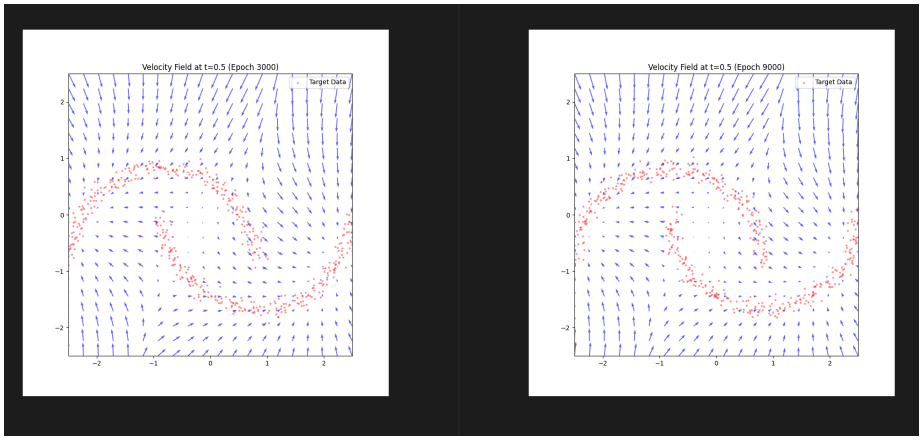


Figure 1: **Learned Velocity Field (Teacher)**. The arrows correctly identify the bifurcation of the dynamics towards the two moons.

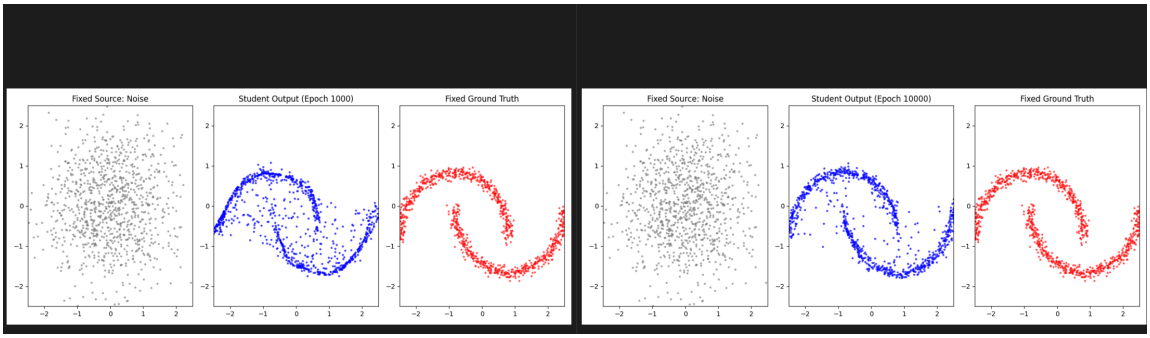


Figure 2: **Direct Transport (Student)**. Left: Source Noise. Center: Predicted Transport at $t = 1$. Right: Ground Truth. The model successfully maps noise to the complex data manifold in a single step.

## 5.3 Training Dynamics

To validate the stability of our training pipeline, we monitored the loss evolution.

During the first stage, the Teacher network (velocity field) converged to a stable plateau corresponding to the noise variance (approx. 1.6 in the stochastic regime).

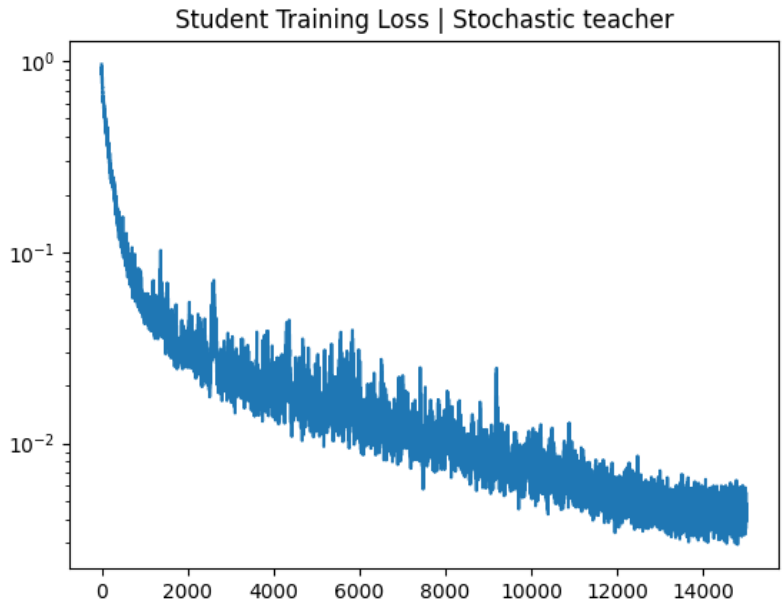


Figure 3: **Student Convergence (L-FMM Loss).** The student's loss decreases rapidly as it learns to distill the teacher's vector field into a direct map. The asymptotic stability indicates that the student has successfully captured the average trajectory of the flow.

# 6 Conclusion

We have successfully replicated the Flow Map Matching method. The Student model achieves generation quality comparable to the Teacher (ODE Solver) but operates orders of magnitude faster (1 function evaluation vs 100). The validation metrics confirm that the learned map respects the underlying semi-group structure of the flow.

# References

[1] Boffi, N. M., Albergo, M. S., & Vanden-Eijnden, E. (2025). *Flow map matching with stochastic interpolants: A mathematical framework for consistency models*. arXiv preprint arXiv:2406.07507.

[2] Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., & Le, M. (2023). *Flow Matching for Generative Modeling*. International Conference on Learning Representations (ICLR).

[3] Liu, X., Gong, C., & Liu, Q. (2022). *Flow Straight and Fast: Learning to Generate with One-Step Flow Matching*. arXiv preprint arXiv:2209.03003.

[4] Albergo, M. S., & Vanden-Eijnden, E. (2023). *Building Normalizing Flows with Stochastic Interpolants*. International Conference on Learning Representations (ICLR).

[5] Brenier, Y. (1991). *Polar factorization and monotone rearrangement of vector-valued functions*. Communications on Pure and Applied Mathematics, 44(4), 375-417.